

[Open in app](#)Artificial INTEL-lige... · [Follow publication](#) Member-only story

Are you too a Poor-GPU-guy? Here's how to run 400B parameter Models for free

A complete guide to NVIDIA NIM's free tier: get hundreds of API calls, access frontier models like Llama 3.3 and DeepSeek, and build a CLI chat app in minutes

16 min read · Mar 31, 2026



Fabio Matricardi

[Follow](#)

Listen



Share

 More

image created with Qwen-image

If you read my articles, I always try to find free API for LLMs. I am a GPU poor guy, so I am always limited to Small Language Models.

I believe you can relate too.

Small Language Models are useful, and honestly for a routine work, study and hobbies are more than enough. A 4B model, or if you have enough RAM even a 7B or 8B can do everything.

But when it comes to Opencode, or any of the Claw series (pawClaw, redClaw... whateverClaw), or even good AI powered web searches, the token count and compute are heavy: and on my Laptop a web search can take up to 35 minutes.

With my fan always humming, and the CPU at 100%.

A free API call can save the day, without touching your budget: not even a cent.

In fact, when you are not working on sensitive data, documents or drawings, a remote call is safe... and faster.

And today I will show you a different service from OpenRouter to do so!

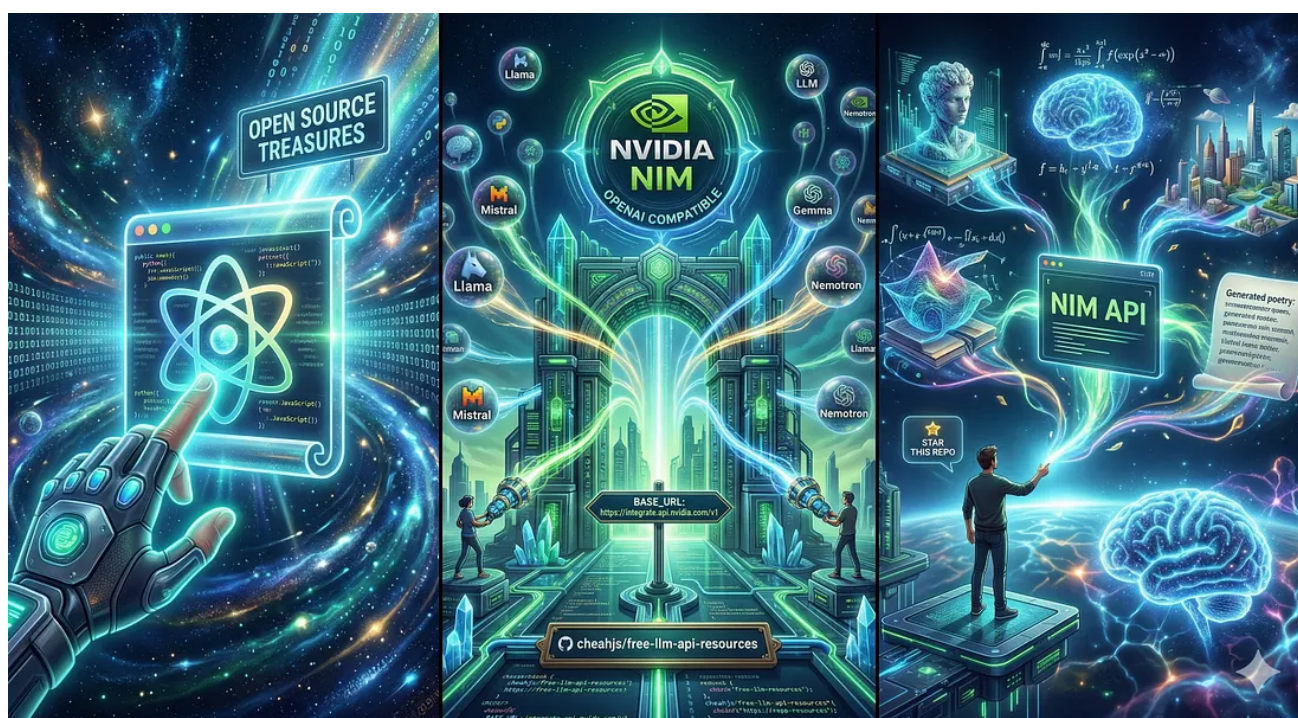
Note: few days ago I talked to you about Perplexica.

Your AI your rules: a free fully local Perplexity app on your PC

How to use Perplexica with llama.cpp server and forget about any monthly fee

blog.stackademic.com

You can point to NVIDIA NIM free Endpoint directly in the app. Try and let me know!



The golden reference

It all started with one of my best bookmarks from Github:

GitHub - cheahjs/free-llm-api-resources: A list of free LLM inference resources accessible via API.

A list of free LLM inference resources accessible via API. - cheahjs/free-llm-api-resources

github.com

The [cheahjs/free-llm-api-resources](https://github.com/cheahjs/free-llm-api-resources) repository is an excellent community-driven directory that highlights various methods for accessing high-performance Large Language Models (LLMs) without upfront costs. Among its featured providers, like OpenRouter, the **NVIDIA NIM** (NVIDIA Inference Microservices) stands out as a premier choice for developers looking to tap into enterprise-grade infrastructure.

On this page

How It Works

Why NIM?

NIM Examples and Blueprints

Getting Started

Learning Library

More Resources



Try

Get free access to NIM API endpoints for unlimited prototyping, powered by DGX Cloud. Your membership to the [NVIDIA Developer Program](#) enables NVIDIA-hosted NIM APIs and containers for development and testing ([FAQ](#)).

[Visit the NVIDIA API Catalog >](#)



Build

Get a head start on development with sample applications built with NIM and partner microservices. [NVIDIA Blueprints](#) can be deployed in one click with [NVIDIA Launchables](#), downloaded for local deployments on PCs and workstations, or for development in your datacenter or private cloud.

[Explore NVIDIA Blueprints >](#)



Deploy

Deploy on your own infrastructure for development and testing. When ready for production, get the assurance of security, API stability, and support that comes with [NVIDIA AI Enterprise](#), or access dedicated enterprise-grade NIM endpoints at NVIDIA partners.

[Run NVIDIA NIM anywhere >](#)

The NVIDIA's hosted API catalog gives us a wide array of state-of-the-art models — ranging from Llama and Mistral to NVIDIA's own Nemotron — all optimized for high-throughput and low-latency performance on specialized GPU hardware.

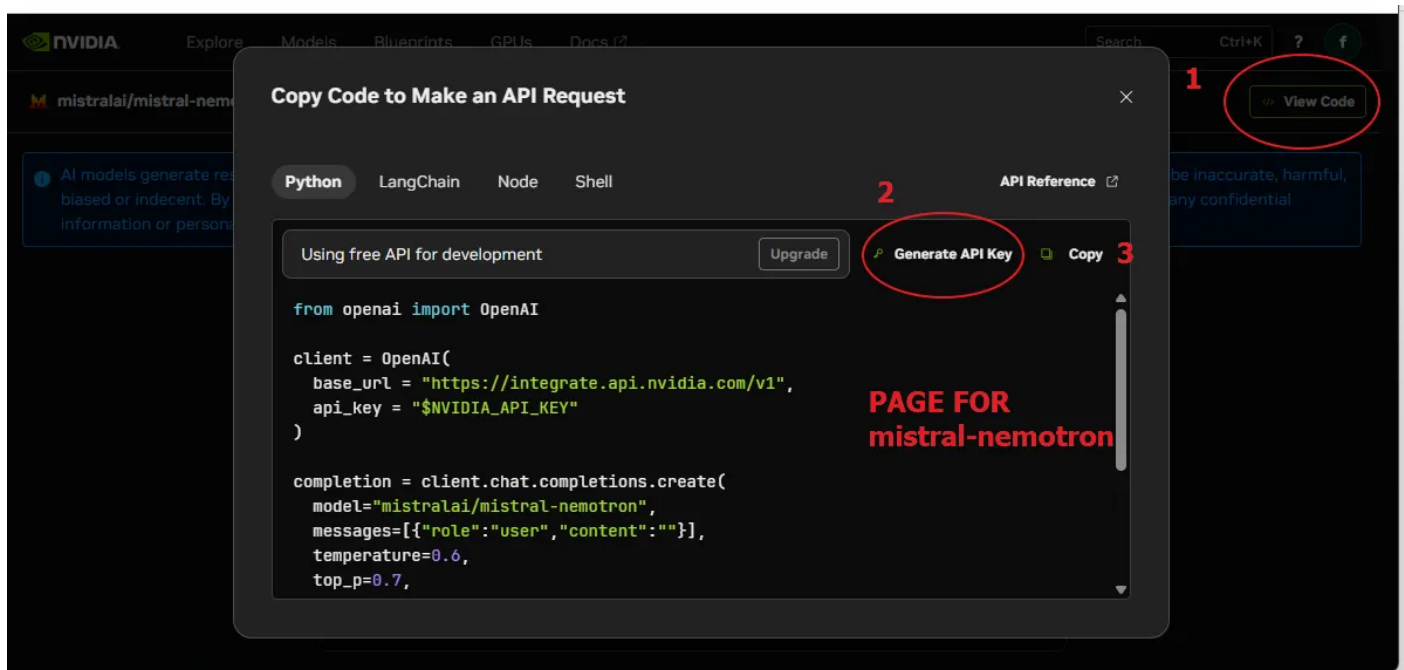
To get started with these free endpoints, you first need to visit the [NVIDIA API Catalog](#) and sign up for an account. When joining the [NVIDIA Developer Program](#), users are typically granted a generous allocation of free credits (often starting with 1,000) to explore and prototype their applications. Once logged in, you can browse the model library, select a specific LLM, and generate an API key. This key acts as your gateway to the service, allowing you to make requests directly to NVIDIA's hosted infrastructure without having to manage any local hardware or complex backend setups.

Phone number verification required. Models tend to be context window limited.

Limits: 40 requests/minute

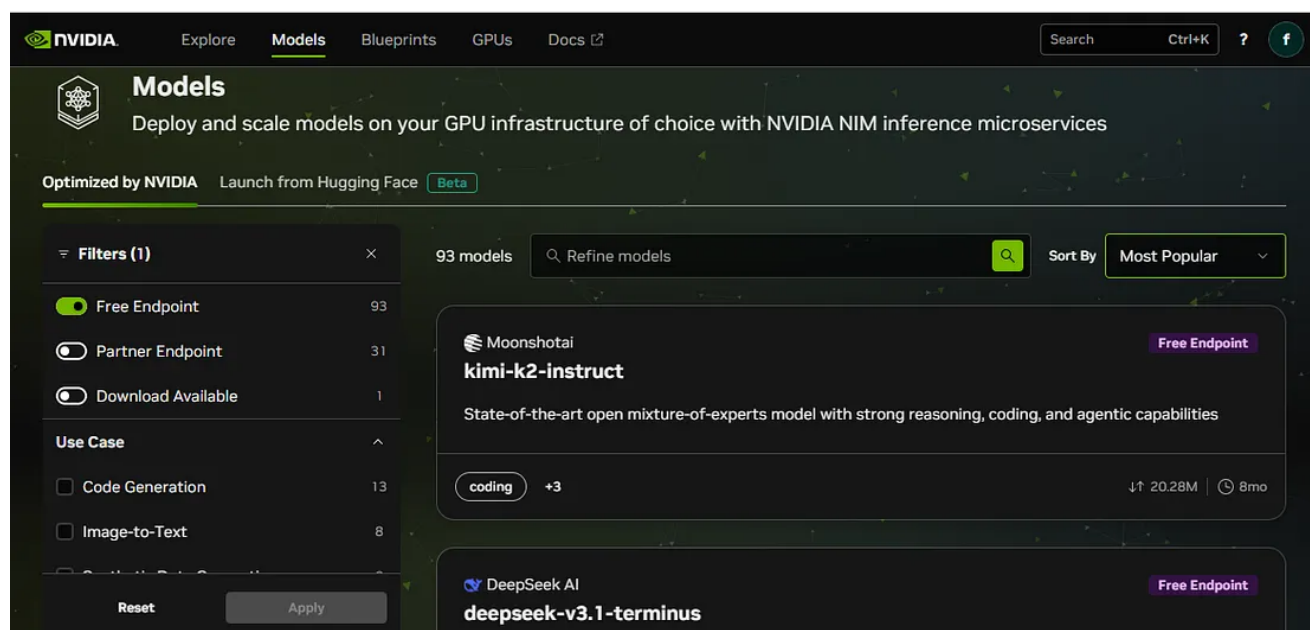
Integrating these endpoints into your project is so easy and straightforward thanks to their **OpenAI-compatible API structure**.

You even get a ready to use snippet (in different languages) and you can generate an API key on the fly in the model card itself!



page for mistral-nemotron from [source](#)

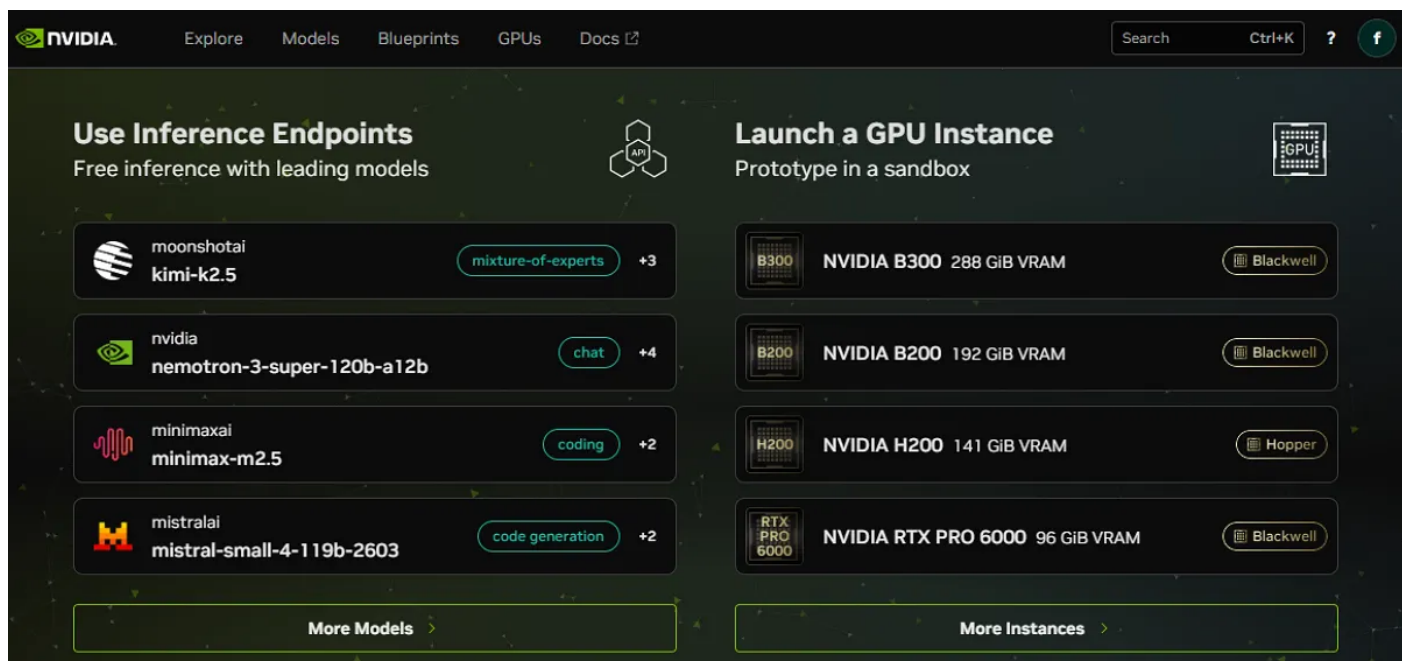
Because the NIM endpoints follow industry standards, you can use popular libraries like the `openai` Python package or standard `curl` commands by simply swapping the `base_url` to `https://integrate.api.nvidia.com/v1`.



screenshot from [NVIDIA Nim free models](#) filter them on the left by **Free Endpoint**

This compatibility ensures that developers can switch from other providers to NVIDIA with minimal code changes, making it an ideal solution for those who want to transition from local prototyping to scalable, GPU-accelerated inference

while maintaining a cost-effective development workflow.



<https://build.nvidia.com/>

NVIDIA micro-services

NVIDIA NIM (NVIDIA Inference Microservices) is a set of easy-to-use microservices designed to accelerate the deployment of generative AI models across any NVIDIA-accelerated infrastructure. By visiting build.nvidia.com, developers can access a massive catalog of enterprise-grade AI models — including Llama 3, Mistral, and NVIDIA's own Nemotron — running on high-performance NVIDIA GPUs.

The Free Tier: how it works

NVIDIA offers a “try-before-you-buy” model designed for prototyping and research. Here is how the free access is structured:

- **Initial Credits:** Upon signing up for the NVIDIA Developer Program, you are typically granted **1,000 free API credits**. These credits allow you to make requests to the hosted NIM endpoints without needing your own GPU hardware.
- **Requesting More:** If you exhaust your initial 1,000 credits, you can often

request additional credits (bringing the total up to 5,000) by providing a business email. This upgrade also typically includes a **90-day NVIDIA AI Enterprise license**, which allows for deeper evaluation of the platform.

- **Transition to Rate Limits:** Recent reports from mid-2025 indicate that NVIDIA has been shifting some users from a strict credit-based system to a **rate-limited model** for ongoing development. This allows for continuous prototyping (roughly **40 requests per minute**) as long as the usage is strictly for non-production, developmental purposes.

Key features of the Free catalog

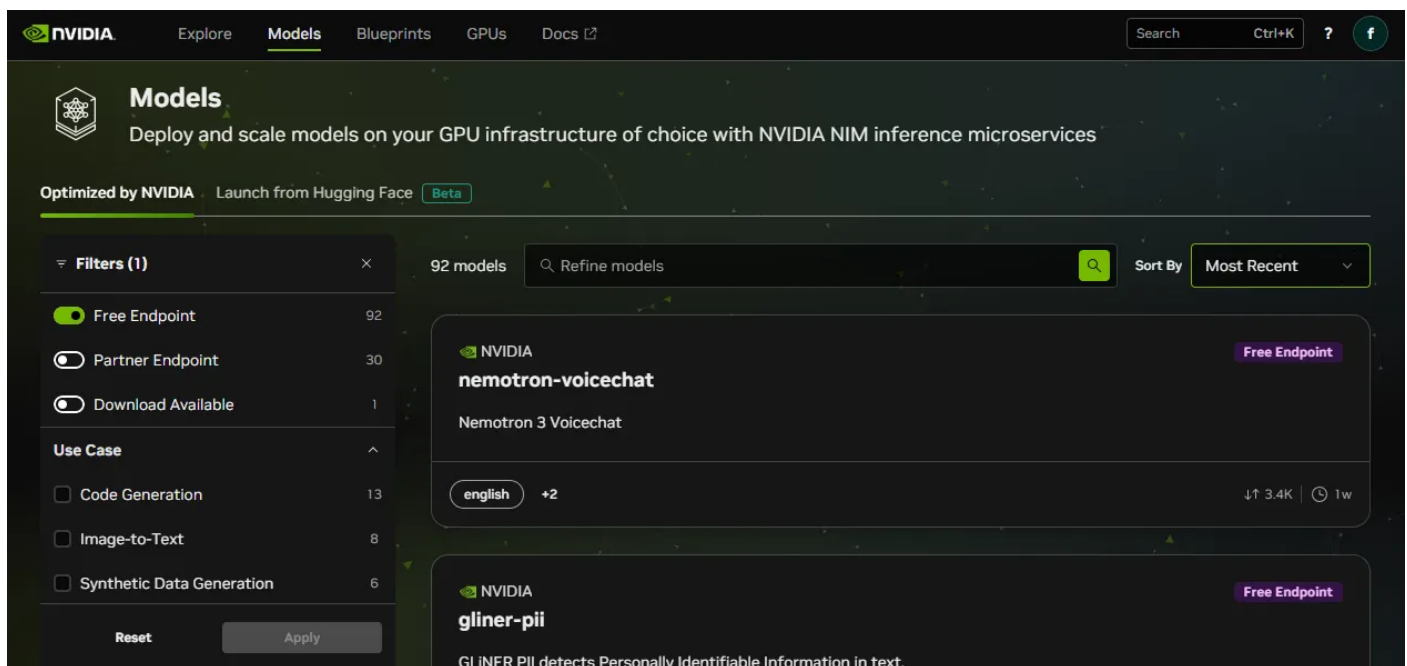
1. **OpenAI-Compatible API:** All models in the catalog follow the standard OpenAI API format. You can integrate them into existing applications by simply changing the `base_url` to NVIDIA's endpoint and providing your API key.
2. **Model Diversity:** You aren't limited to just text LLMs. The catalog includes specialized models for:
 - **Reasoning and Chat:** Llama 3.3, Mistral, and Nemotron.
 - **Vision and Multimodal:** Models capable of processing images and video.
 - **Coding:** Specialized models like DeepSeek-Coder and Qwen-Coder.
 - **BioNeMo:** Specialized models for drug discovery and protein folding.
3. **Portability:** A unique feature of NIM is that once you've prototyped using the free cloud API, you can **download the same NIM container** to run on your own local NVIDIA hardware (RTX PCs or workstations) or private cloud.

Of course... there are some usage restrictions

While the API is free to use for development, NVIDIA distinguishes between

“Development” and “Production”:

- **Free/Developer Tier:** Ideal for students, researchers, and hobbyists building proof-of-concepts. You get access for the duration of your Developer Program membership.
- **NVIDIA AI Enterprise:** Required once you move to production. This provides enterprise-grade security, dedicated support, and higher stability for commercial applications.



free endpoint in alphabetical order

There are, updated on March 2026, 93 models with free API calls

And many frontier models, for text generation, tools call, agentic capabilities and long context.

Here a python script to list all of them (but to know if they have a Free Endpoint... you need to go to the web page model card on NVIDIA NIM).

⚠ To use this python app you need your own API key (see next session)

```
import os
import requests
from openai import OpenAI
```

```
# 1. Using the OpenAI Library (Cleanest method)
client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key="nvapi-you_API-key"
)

def list_nvidia_models():
    models = client.models.list()

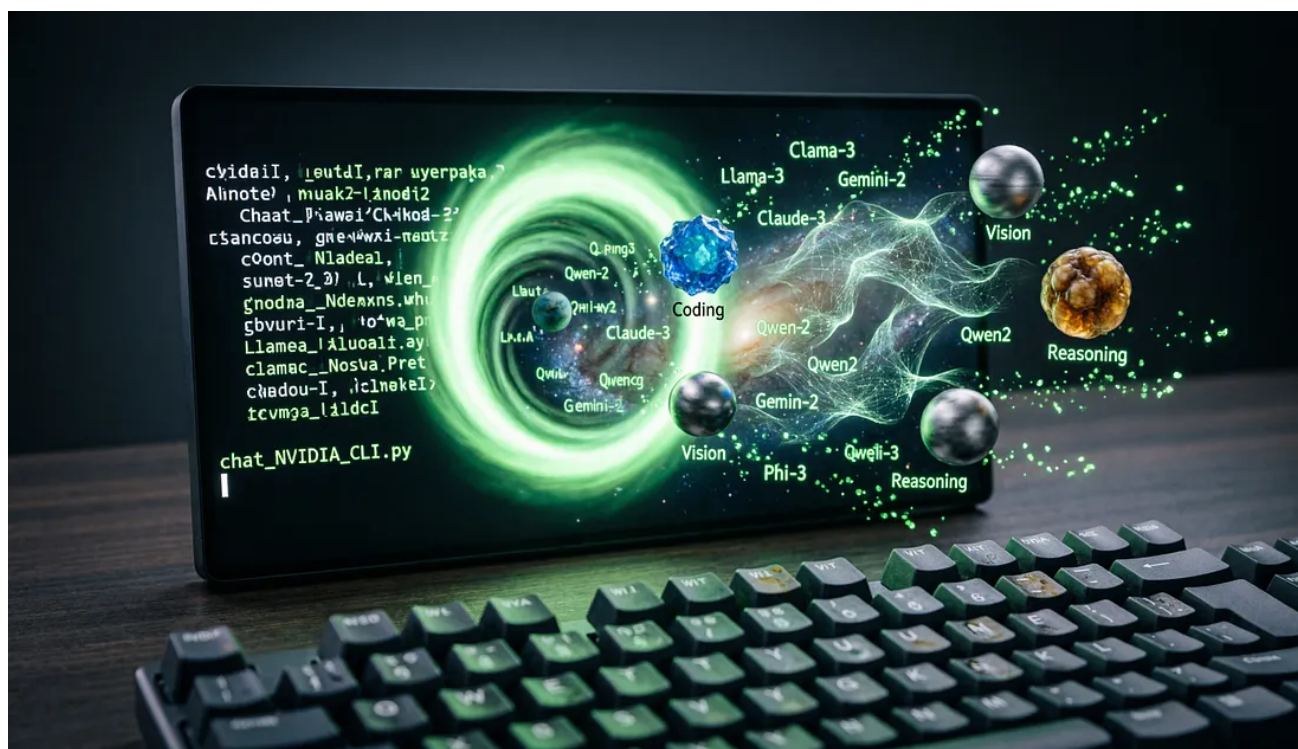
    print(f"{'Model ID':<50} | {'Owned By'}")
    print("-" * 70)

    for model in models.data:
        # Most models in the hosted catalog are part of the 'Free' developer
        print(f"{'model.id':<50} | {'model.owned_by'}")

# 2. Using Requests (Best for raw data/filtering)
def get_raw_model_list():
    url = "https://integrate.api.nvidia.com/v1/models"
    headers = {
        "Authorization": "Bearer your_nvidia_api_key_here",
        "Accept": "application/json"
    }

    response = requests.get(url, headers=headers)
    if response.status_code == 200:
        data = response.json()
        # You can filter the 'data' list here based on naming conventions
        # e.g., models with "instruct" or specific versions
        return data, data['data']
    return []

list_nvidia_models()
```



As you can see yourself, we have a lot of models:

Model ID	Owned By
01-ai/yi-large	01-ai
abacusai/dracarys-llama-3.1-70b-instruct	abacusai
adept/fuyu-8b	adept
ai21labs/jamba-1.5-large-instruct	ai21labs
ai21labs/jamba-1.5-mini-instruct	ai21labs
aisingapore/sea-lion-7b-instruct	aisingapore
baai/bge-m3	baai
baichuan-inc/baichuan2-13b-chat	baichuan-inc
bigcode/starcoder2-15b	bigcode
bigcode/starcoder2-7b	bigcode
bytedance/seed-oss-36b-instruct	bytedance
databricks/dbrx-instruct	databricks
deepseek-ai/deepseek-coder-6.7b-instruct	deepseek-ai
deepseek-ai/deepseek-r1-distill-llama-8b	deepseek-ai
deepseek-ai/deepseek-r1-distill-qwen-14b	deepseek-ai
deepseek-ai/deepseek-r1-distill-qwen-32b	deepseek-ai
deepseek-ai/deepseek-r1-distill-qwen-7b	deepseek-ai
deepseek-ai/deepseek-v3.1	deepseek-ai
deepseek-ai/deepseek-v3.1-terminus	deepseek-ai
deepseek-ai/deepseek-v3.2	deepseek-ai
google/codegemma-1.1-7b	google
google/codegemma-7b	google
google/deplot	google

google/gemma-2-27b-it	google
google/gemma-2-2b-it	google
google/gemma-2-9b-it	google
google/gemma-2b	google
google/gemma-3-12b-it	google
google/gemma-3-1b-it	google
google/gemma-3-27b-it	google
google/gemma-3-4b-it	google
google/gemma-3n-e2b-it	google
google/gemma-3n-e4b-it	google
google/gemma-7b	google
google/paligemma	google
google/recurrentgemma-2b	google
google/shieldgemma-9b	google
gotocompany/gemma-2-9b-cpt-sahabatai-instruct	gotocompany
ibm/granite-3.0-3b-a800m-instruct	ibm
ibm/granite-3.0-8b-instruct	ibm
ibm/granite-3.3-8b-instruct	ibm
ibm/granite-34b-code-instruct	ibm
ibm/granite-8b-code-instruct	ibm
ibm/granite-guardian-3.0-8b	ibm
igenius/colosseum_355b_instruct_16k	igenius
igenius/italia_10b_instruct_16k	igenius
marin/marin-8b-instruct	marin
mediatek/breeze-7b-instruct	mediatek
meta/codellama-70b	meta
meta/llama-3.1-405b-instruct	meta
meta/llama-3.1-70b-instruct	meta
meta/llama-3.1-8b-instruct	meta
meta/llama-3.2-11b-vision-instruct	meta
meta/llama-3.2-1b-instruct	meta
meta/llama-3.2-3b-instruct	meta
meta/llama-3.2-90b-vision-instruct	meta
meta/llama-3.3-70b-instruct	meta
meta/llama-4-maverick-17b-128e-instruct	meta
meta/llama-4-scout-17b-16e-instruct	meta
meta/llama-guard-4-12b	meta
meta/llama2-70b	meta
meta/llama3-70b-instruct	meta
meta/llama3-8b-instruct	meta
microsoft/kosmos-2	microsoft
microsoft/phi-3-medium-128k-instruct	microsoft
microsoft/phi-3-medium-4k-instruct	microsoft
microsoft/phi-3-mini-128k-instruct	microsoft
microsoft/phi-3-mini-4k-instruct	microsoft
microsoft/phi-3-small-128k-instruct	microsoft
microsoft/phi-3-small-8k-instruct	microsoft
microsoft/phi-3-vision-128k-instruct	microsoft
microsoft/phi-3.5-mini-instruct	microsoft
microsoft/phi-3.5-moe-instruct	microsoft
microsoft/phi-3.5-vision-instruct	microsoft
microsoft/phi-4-mini-flash-reasoning	microsoft

microsoft/phi-4-mini-instruct	microsoft
microsoft/phi-4-multimodal-instruct	microsoft
minimaxai/minimax-m2.5	minimaxai
mistralai/codestral-22b-instruct-v0.1	mistralai
mistralai/devstral-2-123b-instruct-2512	mistralai
mistralai/magistral-small-2506	mistralai
mistralai/mamba-codestral-7b-v0.1	mistralai
mistralai/mathstral-7b-v0.1	mistralai
mistralai/ministral-14b-instruct-2512	mistralai
mistralai/mistral-7b-instruct-v0.2	mistralai
mistralai/mistral-7b-instruct-v0.3	mistralai
mistralai/mistral-large	mistralai
mistralai/mistral-large-2-instruct	mistralai
mistralai/mistral-large-3-675b-instruct-2512	mistralai
mistralai/mistral-medium-3-instruct	mistralai
mistralai/mistral-nemotron	mistralai
mistralai/mistral-small-24b-instruct	mistralai
mistralai/mistral-small-3.1-24b-instruct-2503	mistralai
mistralai/mistral-small-4-119b-2603	mistralai
mistralai/mixtral-8x22b-instruct-v0.1	mistralai
mistralai/mixtral-8x22b-v0.1	mistralai
mistralai/mixtral-8x7b-instruct-v0.1	mistralai
moonshotai/kimi-k2-instruct	moonshotai
moonshotai/kimi-k2-instruct-0905	moonshotai
moonshotai/kimi-k2-thinking	moonshotai
moonshotai/kimi-k2.5	moonshotai
nv-mistralai/mistral-nemo-12b-instruct	nv-mistralai
nvidia/cosmos-reason2-8b	nvidia
nvidia/embed-qa-4	nvidia
nvidia/gliner-pii	nvidia
nvidia/llama-3.1-nemoguard-8b-content-safety	nvidia
nvidia/llama-3.1-nemoguard-8b-topic-control	nvidia
nvidia/llama-3.1-nemotron-51b-instruct	nvidia
nvidia/llama-3.1-nemotron-70b-instruct	nvidia
nvidia/llama-3.1-nemotron-70b-reward	nvidia
nvidia/llama-3.1-nemotron-nano-4b-v1.1	nvidia
nvidia/llama-3.1-nemotron-nano-8b-v1	nvidia
nvidia/llama-3.1-nemotron-nano-vl-8b-v1	nvidia
nvidia/llama-3.1-nemotron-safety-guard-8b-v3	nvidia
nvidia/llama-3.1-nemotron-ultra-253b-v1	nvidia
nvidia/llama-3.2-nemoretriever-1b-vlm-embed-v1	nvidia
nvidia/llama-3.2-nemoretriever-300m-embed-v1	nvidia
nvidia/llama-3.2-nv-embedqa-1b-v1	nvidia
nvidia/llama-3.2-nv-embedqa-1b-v2	nvidia
nvidia/llama-3.3-nemotron-super-49b-v1	nvidia
nvidia/llama-3.3-nemotron-super-49b-v1.5	nvidia
nvidia/llama-nemotron-embed-1b-v2	nvidia
nvidia/llama-nemotron-embed-vl-1b-v2	nvidia
nvidia/llama3-chatqa-1.5-70b	nvidia
nvidia/llama3-chatqa-1.5-8b	nvidia
nvidia/mistral-nemo-minitron-8b-8k-instruct	nvidia
nvidia/mistral-nemo-minitron-8b-base	nvidia

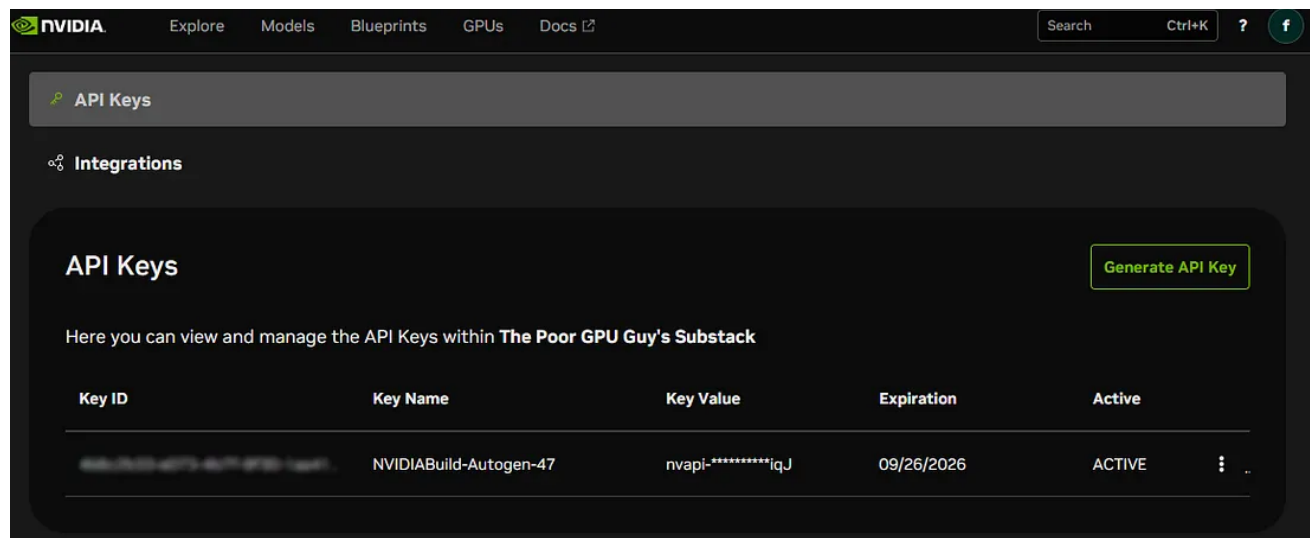
nvidia/nemoretriever-parse	nvidia
nvidia/nemotron-3-nano-30b-a3b	nvidia
nvidia/nemotron-3-super-120b-a12b	nvidia
nvidia/nemotron-4-340b-instruct	nvidia
nvidia/nemotron-4-340b-reward	nvidia
nvidia/nemotron-4-mini-hindi-4b-instruct	nvidia
nvidia/nemotron-content-safety-reasoning-4b	nvidia
nvidia/nemotron-mini-4b-instruct	nvidia
nvidia/nemotron-nano-12b-v2-v1	nvidia
nvidia/nemotron-nano-3-30b-a3b	nvidia
nvidia/nemotron-parse	nvidia
nvidia/neva-22b	nvidia
nvidia/nv-embed-v1	nvidia
nvidia/nv-embedcode-7b-v1	nvidia
nvidia/nv-embedqa-e5-v5	nvidia
nvidia/nv-embedqa-mistral-7b-v2	nvidia
nvidia/nvclip	nvidia
nvidia/nvidia-nemotron-nano-9b-v2	nvidia
nvidia/riva-translate-4b-instruct	nvidia
nvidia/riva-translate-4b-instruct-v1.1	nvidia
nvidia/streampetr	nvidia
nvidia/usdcode-llama-3.1-70b-instruct	nvidia
nvidia/vila	nvidia
openai/gpt-oss-120b	openai
openai/gpt-oss-120b	openai
openai/gpt-oss-20b	openai
openai/gpt-oss-20b	openai
opengpt-x/teuken-7b-instruct-commercial-v0.4	opengpt-x
qwen/qwen2-7b-instruct	qwen
qwen/qwen2.5-7b-instruct	qwen
qwen/qwen2.5-coder-32b-instruct	qwen
qwen/qwen2.5-coder-7b-instruct	qwen
qwen/qwen3-coder-480b-a35b-instruct	qwen
qwen/qwen3-next-80b-a3b-instruct	qwen
qwen/qwen3-next-80b-a3b-thinking	qwen
qwen/qwen3.5-122b-a10b	qwen
qwen/qwen3.5-397b-a17b	qwen
qwen/qwq-32b	qwen
rakuten/rakutenai-7b-chat	rakuten
rakuten/rakutenai-7b-instruct	rakuten
sarvamai/sarvam-m	sarvamai
snowflake/arctic-embed-l	snowflake
speakleash/bielik-11b-v2.3-instruct	speakleash
speakleash/bielik-11b-v2.6-instruct	speakleash
stepfun-ai/step-3.5-flash	stepfun-ai
stockmark/stockmark-2-100b-instruct	stockmark
thudm/chatglm3-6b	thudm
tiiuae/falcon3-7b-instruct	tiiuae
tokyotech-llm/llama-3-swallow-70b-instruct-v0.1	tokyotech-llm
upstage/solar-10.7b-instruct	upstage
utter-project/eurollm-9b-instruct	utter-project
writer/palmyra-creative-122b	writer

writer/palmyra-fin-70b-32k	writer
writer/palmyra-med-70b	writer
writer/palmyra-med-70b-32k	writer
yentinglin/llama-3-taiwan-70b-instruct	yentinglin
z-ai/glm4.7	z-ai
z-ai/glm5	z-ai
zyphra/zamba2-7b-instruct	zyphra

Get your API calls

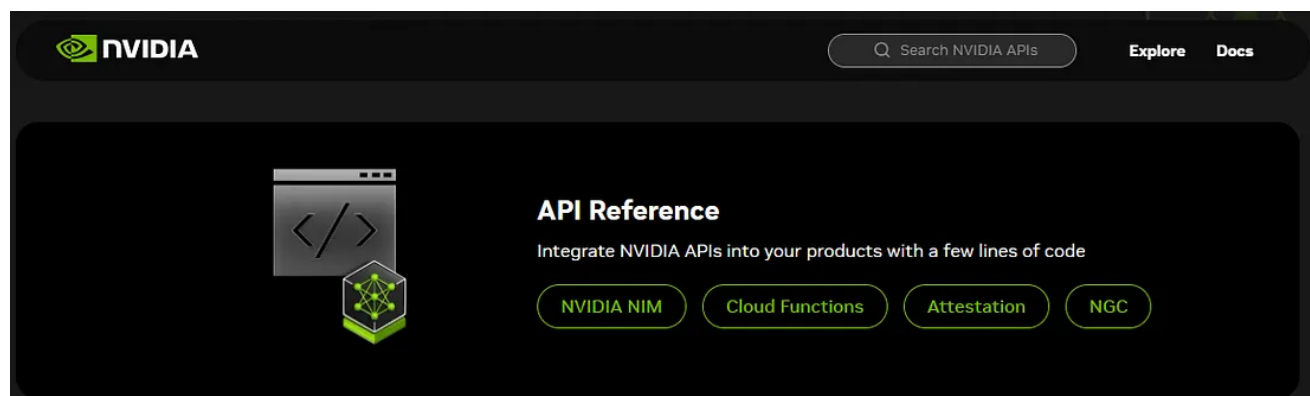
Register from <https://build.nvidia.com/> or from <https://developer.nvidia.com/nim>. Confirm the email, and verify your mobile number (you will get a code). And you are ready to go!

In your profile add an API key (and save it somewhere on your PC!!!)



The Docs are well curated and clear. But in case you cannot understand them, paste the link in any AI powered search engine and ask for clarification.

I did the same, in few cases.



read the docs — <https://docs.api.nvidia.com/>

Let's build a quick CLI chat application, that will save the conversation history, using a reasoning model for free from NVIDIA NIM.

Packages

We don't need a virtual environment, because these libraries are used everywhere:

```
pip install rich requests openai
```

The chat app (in the terminal)

This Python tool transforms a basic API connection into a polished, terminal-based chat application.

If you are writing about free or high-performance LLM providers, NVIDIA is a standout because of its **speed** and the **variety of open-weight models** (like Llama 3, Mistral, and DeepSeek) available to test for free through their Integrate API.

🌟 Here the features I created at a glance

- **Interactive Model Selection:** Choose your “brain” from a list of top-tier AI models at startup.
- **Memory (Context Persistence):** The app remembers what you said 10 minutes ago, allowing for true back-and-forth conversations.
- **Thinking Mode:** Specialized support for “Reasoning” models (like DeepSeek or GLM) where you can see the AI's “inner monologue” before it answers.
- **Smart Session Logging:** Every chat is automatically saved to a unique `.json` file. You can stop a chat today and resume it tomorrow!
- **Markdown Support:** Code blocks, bold text, and lists are rendered beautifully in your terminal.

- **Token Tracker:** Real-time stats on how much “data” (tokens) your conversation is consuming.

```
import os
import json
import glob
from datetime import datetime
from openai import OpenAI
from rich.console import Console
from rich.markdown import Markdown
from rich.prompt import Prompt
from rich.panel import Panel
from rich.table import Table

console = Console()

MODELS = [
    "stepfun-ai/step-3.5-flash",
    "z-ai/glm4.7",
    "moonshotai/kimi-k2-thinking",
    "mistralai/mistral-nemotron",
    "marin/marin-8b-instruct"
]

client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key="nvapi-yourAPIkey"
)

def get_multiline_input():
    """Allows user to paste or type multiple lines. Ends on Ctrl+D (Unix) or
    console. print("[bold green]USER (Ctrl+D/Z to send, 'exit' to quit):[/bold
    lines = []
    try:
        while True:
            line = input()
            lines.append(line)
    except EOFError:
        pass
    return "\n".join(lines).strip()

def select_session():
    """Scans for existing logs and asks to resume or start new."""
    files = glob.glob("chat_log*.json")
    if not files:
        return None, []

    console.print(Panel("[bold yellow]Existing Sessions Found[/bold yellow]"
    console.print("0: [bold green]Start a New Session[/bold green]")
```

```

for i, f in enumerate(files, 1):
    console.print(f"{i}: {f}")

choice = Prompt.ask("\nChoose a session", choices=[str(i) for i in range(1, len(files) + 1)])

if choice == "0":
    return None, []

filename = files[int(choice) - 1]
with open(filename, "r") as f:
    return filename, json.load(f)

def select_model():
    console.print(Panel("[bold blue]Model Selection[/bold blue]", expand=False))
    for i, m in enumerate(MODELS, 1):
        console.print(f"[bold cyan]{i}[/bold cyan]: {m}")
    choice = Prompt.ask("\nSelect model", choices=[str(i) for i in range(1, len(MODELS) + 1)])
    return MODELS[int(choice) - 1]

def main():
    # 1. Session & Model Setup
    log_filename, history = select_session()
    if not log_filename:
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        log_filename = f"chat_log_{timestamp}.json"
        history = [{"role": "system", "content": "You are a helpful assistant"}]

    selected_model = select_model()
    console.print(f"\n[bold yellow]Active Session:[/bold yellow] [green]{log_filename}")

    try:
        while True:
            user_input = get_multiline_input()
            if not user_input: continue
            if user_input.lower() in ["exit", "quit"]: break

            history.append({"role": "user", "content": user_input})

            # API Call with Usage Tracking
            completion = client.chat.completions.create(
                model=selected_model,
                messages=history,
                stream=True,
                stream_options={"include_usage": True}, # Crucial for token counting
                extra_body={"chat_template_kwargs": {"enable_thinking": True}}
            )

            full_content = ""
            is_thinking = False
            usage_data = None

            for chunk in completion:

```

```

# Handle Usage Chunk (usually the last chunk)
if hasattr(chunk, 'usage') and chunk.usage is not None:
    usage_data = chunk.usage
    continue

if not chunk.choices: continue
delta = chunk.choices[0].delta

# Reasoning
reasoning = getattr(delta, "reasoning_content", None)
if reasoning:
    if not is_thinking:
        console.print("\n[bold magenta]■ THINKING[/bold magenta]
        is_thinking = True
        console.print(f"[grey50]{reasoning}[/grey50]", end="", h

# Content
if getattr(delta, "content", None) is not None:
    if is_thinking:
        console.print("\n\n[bold cyan]■ RESPONSE[/bold cyan]
        is_thinking = False
        console.print(delta.content, end="", highlight=False)
        full_content += delta.content

# Turn Summary
if full_content:
    console.print("\n" + "-" * console.width)
    console.print(Markdown(full_content))

# Token Usage Summary
if usage_data:
    table = Table(title="Token Usage", show_header=False, bo
    table.add_row("Prompt Tokens", f"[cyan]{usage_data.promp
    table.add_row("Completion Tokens", f"[cyan]{usage_data.c
    table.add_row("Total Tokens", f"[bold green]{usage_data.
    console.print(table)

    console.print("-" * console.width + "\n")
    history.append({"role": "assistant", "content": full_content

    with open(log_filename, "w") as f:
        json.dump(history, f, indent=4)

except KeyboardInterrupt:
    console.print("\n[bold red]Interrupted. Saving and exiting...[/bold
finally:
    console.print(f"[bold yellow]History saved to {log_filename}[/bold y

if __name__ == "__main__":
    main()

```

```

3. **Controversy & Consequences**: The US justified the war as a preemptive strike to prevent Iranian aggression and secure regional stability, but it was widely condemned as illegal, with the UN and various experts criticizing it as a violation of international law.

Medium Draft Saved Summary: Publish
On 28 February 2026, the United States and Israel launched a surprise attack on Iran, killing Supreme Leader Ali Khamenei and other officials. This triggered a full-scale war, with Iran retaliating with missile and drone strikes against Israel, US bases, and allied countries. The conflict intensified tensions in the region, disrupted global trade (including oil and gas supply chains), and inflicted widespread destruction on civilian and military targets. The US cited reasons like preemptive action, countering Iran's military capabilities, and preventing nuclear proliferation, but the strikes were condemned by many, including the United Nations, as illegal and destabilizing. The war led to significant financial and economic fallout, including military costs for the US reaching $18 billion, followed by a Pentagon request for an additional $200 billion.

3 Key Points:
1 Trigger & Escalation: The war began with a surprise US-Israeli attack on Iran, killing its leader, which led to Iran launching retaliatory strikes across the region, including on US allies.
2 Global Impact: The conflict caused severe economic disruptions, particularly in oil and gas markets, aviation, and financial sectors, reminiscent of the 1970s energy crisis.
3 Controversy & Consequences: The US justified the war as a preemptive strike to prevent Iranian aggression and secure regional stability, but it was widely condemned as illegal, with the UN and various experts criticizing it as a violation of international law.

Token Usage
Prompt Tokens 1298
Completion Tokens 314
Total Tokens 1612

USER (Ctrl+D/Z to send, 'exit' to quit):
exit
^Z
History saved to chat_log_20260327_145510.json
PS C:\tests\NVIDIA_api>

```

the app running

How It Works

You don't need to be a python expert to understand this script. It follows a simple Loop of Life:

1. The Setup (`select_session` & `select_model`)

When you run the script, it first looks around your folder. If it sees old chat files (`chat_log_...json`), it asks if you want to continue an old conversation. Then, it lets you pick which AI model you want to talk to.

2. The Multiline Input (`get_multiline_input`)

Standard chat apps often break if you try to paste a long email or a block of code. This function allows you to paste as much as you want. You simply hit **Ctrl+Z** and **Enter** (on Windows) or **Ctrl+D** (on Linux/Mac) when you're done typing to send the "envelope" to NVIDIA.

3. The Stream (`completion`)

Instead of waiting for the AI to think of the *entire* answer (which can take seconds), we "stream" it. The text appears on your screen character-by-character, just like ChatGPT.

4. The "Thinking" vs "Responding" Logic

Modern AI models often perform “Chain of Thought” reasoning. Our script is smart enough to detect this:

- **Thinking:** Displayed in a dimmed, grey style.
- **Responding:** Displayed in bright, clear Markdown once the thinking is finished.

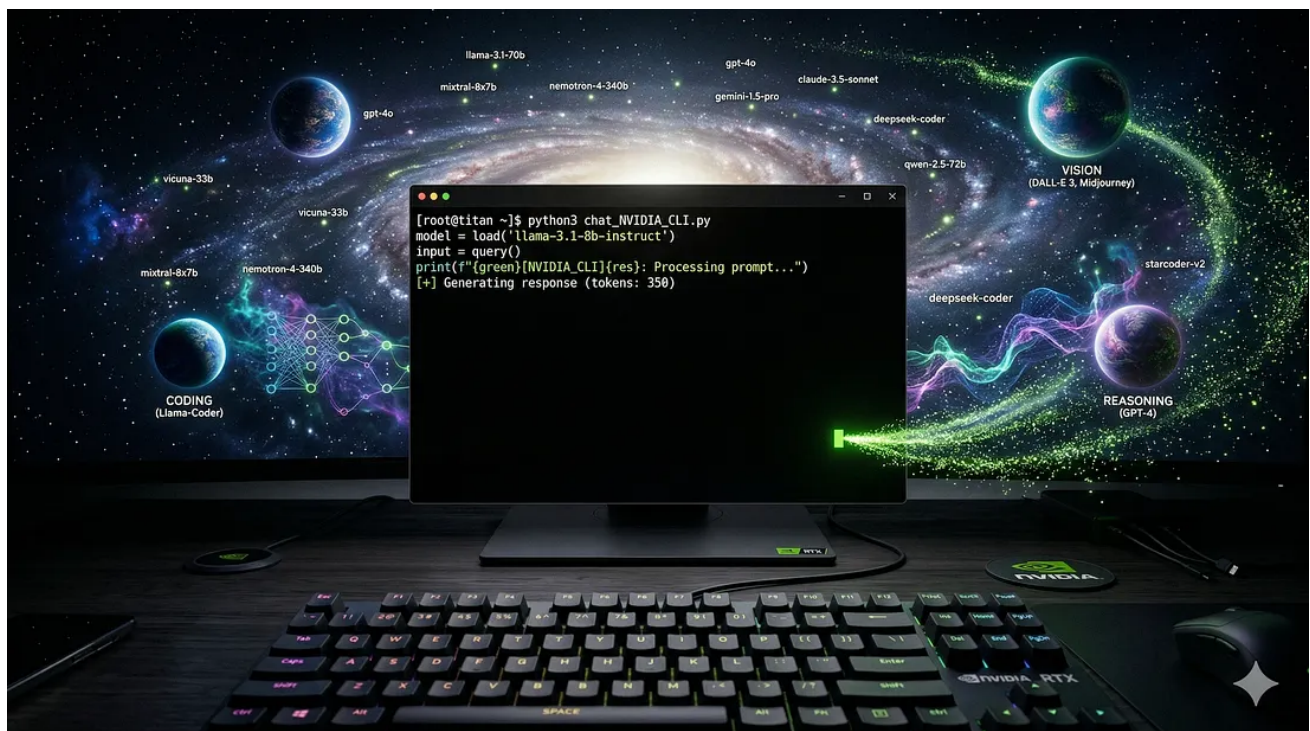


image generated with gemini-flash

I called my python app `chat_NVIDIA_CLI.py`

Running the App

1. Open the script and replace `"nvapi-your_API_key"` with your actual key.
2. Run the script:

```
python chat_NVIDIA_CLI.py
```

The Bottom Line: your Desktop, only smarter

We live in an era where “State of the Art” doesn’t have to mean “State of your Bank Account.” Linking together a simple Python script and NVIDIA’s enterprise-grade NIM endpoints, you’ve essentially given your humble laptop a brain transplant.

You no longer have to listen to your cooling fans scream just to get a decent answer from a 4B model. You can now tap into the 70B and 400B giants for free, keeping your hardware cool and your workflow lightning-fast.

Take the Challenge

Don’t just bookmark this article and let it gather digital dust. Here is your 10-minute challenge:

1. **Grab your API key** from build.nvidia.com.
2. **Copy the script** above into a file named `chat_NVIDIA_CLI.py`.
3. **Run it** and ask a reasoning model like `glm-4` or `kimik2` to explain a complex topic you've been struggling with.

Did it work for you? Which model is currently your favorite? Drop a comment below and let me know if you ran into any snags or if you’ve customized the script further. If you found this helpful, give it a clap (or ten!) so other “GPU-poor” developers can find their way to the NVIDIA treasure chest.

I hope you enjoyed the article. If this story provided value and you wish to show a little support, you could:

1. Clap a lot of times for this story
2. Highlight the parts more relevant to be remembered (it will be easier for you to find them later and for me to write better articles)
3. Join my [totally free weekly Substack newsletter here](#)

4. Follow me on Medium

5. Follow my publication <https://medium.com/artificial-intel-ligence-playground>

If you want to read more, here are some ideas:

Your Tabs are lying to you

I Finally Stopped Believing Them. How I tamed my 70-tab chaos with a 100-line Python script, and finally stopped...

blog.stackademic.com

Chocolatey for Python & AI enthusiasts: how to turn Windows into your in-house developer...

How a terminal-first package manager can automate your tools, tame your installs, and make reproducible AI environments...

ai.gopubby.com

Stop chasing AI benchmarks. Be the Benchmark.

In a world of endless AI releases and shiny leaderboards, the smartest strategy is simple: treat benchmarks as signals...

blog.stackademic.com

llama.CPP restyle is the workshop for your Local AI

From LLM routing to full working Chat application: all in one ZIP file. And here is how.

medium.com

From zero to AI automation: a Python adventure

How to use Python and AI to fight boring tasks... so you don't have to.

medium.com

Thepoorgpuguy

Python4ai

Free Api

Generativeai

Nvidia



Follow

Published in Artificial INTEL-ligence Playground

278 followers · Last published 1 day ago

Hands-On & Hardware-Friendly go-to resource for AI tutorials, llama.cpp, projects, and news, designed for low-hardware setups. Build your local AI, because Your AI your Rules.



Follow

Written by Fabio Matricardi

9.6K followers · 825 following

passionate educator, curious industrial automation engineer. Learning Leadership and how to build my own AI. contact me at fabio.matricardi@gmail.com

